

Métodos	Estático	Dinámico
Apilar()	Temp: $O(n)$ Espac. $O(n)$	
Desapilar()	Temp: $O(1)$ Espac. $O(n)$	Pila: Temp: $O(1)$ Espac. $O(n)$
Tope()	Temp: $O(1)$ Espac. $O(n)$	
Pila Vacía	Temp: $O(1)$ Espac. $O(n)$	
TDA Conjunto	Temp: $O(N)$ Espac. $O(n)$	Temp: $O(n)$ Espac. $O(n)$
TDA Cola	Temp: $O(1)$ Espac. $O(n)$	Temp: $O(1)$ Espac. $O(n)$
TDA Cola Prioridad	Temp: $O(n)$ Espac. $O(n)$	Temp: $O(n)$ Espac. $O(n)$
TDA Diccionario simple	Temp: $O(n)$ Espac. $O(n)$	Temp: $O(n)$ Espac. $O(n)$
TDA Diccionario múltiple	Temp: $O(n)$ Espac. $O(n)$	Temp: $O(n)$ Espac. $O(n)$

Pruebas realizadas en Big-O Calculator Estáticos:

- **TDA Conjunto Estático**

Analysis Result

The provided implementation of the Conjunto class uses an array to store elements and performs various set operations.

Time Complexity:

- InicializarConjunto: $O(1)$ — initializes the array and counter.
- Agregar: $O(n)$ in the worst case, due to the Pertenece check (linear search) before adding.
- Sacar: $O(n)$ — searches for the element to remove, then swaps with the last element.
- Tamano: $O(1)$ — returns the current size.
- Pertenece: $O(n)$ — linear search to check membership.
- ConjuntoVacio: $O(1)$ — checks if cantidad == 0.
- Elegir: $O(1)$ — returns the first element.

Overall, most operations (except Tamano, ConjuntoVacio, Elegir) are linear time due to the use of linear search.

Space Complexity:

- The array datos has a fixed size of MAX (100), so space complexity is $O(1)$ with respect to input size, as the maximum size is constant.
- Additional space is minimal, only a few variables for counters and indices.

In summary, the implementation provides constant space but linear time complexity for most set operations, which is acceptable for small fixed-size sets but may become inefficient for larger or dynamic sets.

- **TDA Cola Estático**

Analysis Result

The implementation of the Cola class uses an array-based circular queue.

Time Complexity:

- InicializarCola(): $O(1)$ – initializes the array and resets indices.
- Acolar(int x): $O(1)$ – inserts an element at the position 'ultimo' and updates the index.
- Desacolar(): $O(1)$ – advances the 'primero' index.
- Primero(): $O(1)$ – retrieves the element at 'primero'.
- ColaVacia(): $O(1)$ – compares 'primero' and 'ultimo'.

Overall, each operation runs in constant time, $O(1)$.

Space Complexity:

- The array 'arr' has a fixed size of MAX (100), so space used is $O(\text{MAX})$.
- Additional variables (primero, ultimo) use constant space.

Thus, the space complexity is $O(\text{MAX})$, which is constant with respect to the number of operations, assuming MAX is fixed.

- **TDA cola con prioridad:**

Analysis Result

The implementation of the ColaPrioridad class uses arrays to store elements and their priorities, with a maximum size defined by MAX (100).

Time Complexity:

- Initialization (InicializarColaPrioridad): $O(1)$, as it simply assigns new arrays and resets the count.
- Enqueue (AcolarPrioridad): $O(1)$, since it inserts at the end without shifting elements.
- Find maximum (Primero and Prioridad): $O(n)$, as they iterate through all elements to find the one with the highest priority.
- Dequeue (Desacolar): $O(n)$, because it searches for the element with the highest priority, then shifts elements to remove it (though shifting isn't explicitly shown, it would be necessary to maintain array integrity).

Space Complexity:

- $O(n)$, where n is the number of elements stored (up to MAX). The arrays valores and prioridades each occupy space proportional to MAX, and the auxiliary variables use constant space.

Overall, the implementation is simple but not optimized for large datasets, as operations like finding the maximum are linear in the number of elements.

- **TDA diccionario simple**

Analysis Result

The implementation of `DiccionarioSimple` uses arrays to store keys and values, with a fixed maximum size ($MAX = 100$).

Time Complexity:

- `InicializarDiccionario`: $O(1)$, as it just initializes arrays and counter.
- `buscarIndice`: $O(n)$, where n is the number of stored elements (up to MAX), since it performs a linear search.
- `Agregar`: $O(n)$ in the worst case, because it calls `buscarIndice` to check if the key exists, which is linear, and then potentially adds a new element.
- `Eliminar`: $O(n)$, due to `buscarIndice` to find the key, and then shifting or replacing the element at the found index.
- `Recuperar`: $O(n)$, as it calls `buscarIndice` to find the key.

Space Complexity:

- Uses arrays of size MAX for storing keys and values, so space complexity is $O(MAX)$. The actual used space depends on the number of stored elements (up to MAX), but the allocated space remains constant.

Overall, the implementation is simple but not optimized for large datasets, as most operations are linear in the number of stored elements.

Pruebas de TDA dinámicos realizadas en Big-O Calculator:

- TDA pila dinámica

Analysis Result

The implementation of the PilaDinamica class provides standard stack operations using a linked list structure.

Time Complexity:

- InicializarPila: $O(1)$, simply sets the tope reference to null.
- Apilar: $O(1)$, creates a new node and updates the tope pointer.
- Desapilar: $O(1)$, updates the tope pointer to the next node.
- Tope: $O(1)$, retrieves the data from the current tope node.
- PilaVacía: $O(1)$, checks if tope is null.

Overall, all operations run in constant time, $O(1)$.

Space Complexity:

- Each element pushed onto the stack allocates a new node, so space grows linearly with the number of elements, $O(n)$.
- The auxiliary space per operation is constant, $O(1)$.

In summary, the implementation provides efficient constant-time operations with linear space growth proportional to the number of elements stored.

- TDA Cola Dinámica

Analysis Result

The implementation of the dynamic queue (ColaDinamica) uses linked nodes, which allows for efficient enqueue and dequeue operations.

Time Complexity:

- InicializarCola: $O(1)$, simply sets references to null.
- Acolar (enqueue): $O(1)$, adds a new node at the end by updating the 'fondo' pointer.
- Desacolar (dequeue): $O(1)$, advances the 'frente' pointer to the next node.
- Primero (peek): $O(1)$, returns data from the 'frente' node.
- ColaVacía: $O(1)$, checks if 'frente' is null.

Space Complexity:

- $O(n)$, where n is the number of elements in the queue, since each element is stored in a node with a reference to the next node. The space used grows linearly with the number of elements.

- **TDA Conjunto dinámico**

Analysis Result

The implementation uses a singly linked list to represent the set.

Time Complexity:

- InicializarConjunto: $O(1)$, simply sets the head to null.
- Agregar: $O(n)$ in the worst case, because it calls Pertenece to check for duplicates, which traverses the list, and then potentially adds at the head.
- Sacar: $O(n)$, as it may need to traverse the entire list to find the element to remove.
- Pertenece: $O(n)$, since it searches through the list to find the element.
- Elegir: $O(1)$, directly returns the data at the head.
- Tamano: $O(n)$, as it traverses the entire list to count elements.

Space Complexity:

- The space used is proportional to the number of elements in the set, $O(n)$, since each element is stored in a node with a reference to the next node. The auxiliary space is constant, $O(1)$, aside from the storage of nodes.

- **TDA Cola con prioridad dinámica:**

Analysis Result

The implementation of the dynamic priority queue (ColaPrioridadDinamica) uses a linked list to maintain elements ordered by priority.

Time Complexity:

- InicializarColaPrioridad: $O(1)$, simply sets the front pointer to null.
- AcolarPrioridad: $O(n)$ in the worst case, because it may need to traverse the entire list to find the correct insertion point, especially when inserting at the end.
- Desacolar: $O(1)$, as it just moves the front pointer to the next node.
- ColaVacía: $O(1)$, checks if the front is null.
- Primero and Prioridad: $O(1)$, access the data directly from the front node.

Space Complexity:

- $O(n)$, where n is the number of elements in the queue, since each element is stored in a node with references to the next node.

Overall, insertion (AcolarPrioridad) is the most costly operation at $O(n)$, while other operations are constant time. The space used scales linearly with the number of elements.

TDA diccionario simple dinámico:

Analysis Result

The implementation uses a singly linked list to store key-value pairs, where each node (NodeClave) contains a key, a reference to a linked list of values (NodeValor), and a pointer to the next NodeClave.

Time Complexity:

- InicializarDiccionario: $O(1)$ – simply sets the head to null.
- Agregar: $O(n)$ in the worst case, where n is the number of entries, because it may need to traverse the entire list to find if the key exists. If the key exists, updating the value is $O(1)$. If not, inserting at the head is $O(1)$.
- Eliminar: $O(n)$ in the worst case, as it may need to traverse the entire list to find the key to delete.
- Recuperar: $O(n)$ in the worst case, as it may need to traverse the list to find the key.
- Claves: $O(n)$, since it traverses the entire list to collect all keys.

Space Complexity:

- $O(n)$ for storing n key-value pairs, with each node containing references to the key, value, and next node. The total space grows linearly with the number of entries.

Overall, this implementation is efficient for small datasets but may become slow as the number of entries increases due to linear search times.

TDA diccionario múltiple dinámico:

Analysis Result

The implementation uses linked lists to store keys and associated values.

Time Complexity:

- Initializing the dictionary (InicializarDiccionario): $O(1)$.
- Adding a key-value pair (Agregar):
 - Searching for the key (buscarClave): $O(n)$, where n is the number of keys.
 - If key not found, inserting at the head: $O(1)$.
 - If key exists and value is new, inserting at the head of the value list: $O(1)$.
- Removing a value (EliminarValor):
 - Searching for the key: $O(n)$.
 - Traversing the value list to find the value: $O(m)$, where m is the number of values for that key.
- Retrieving values (Recuperar):
 - Searching for the key: $O(n)$.
 - Returning the list of values: $O(m)$.

Overall, most operations are linear in the number of keys or values, making the average case $O(n + m)$.

Space Complexity:

- The space used depends on the number of keys and values stored.
- Each key and value is stored in a node, so space complexity is $O(n + \text{total number of values})$, which is proportional to the total entries stored in the dictionary.

Ejercicios 14/5/26:

EJERCICIO 1:

Análisis de ciclos anidados con condición

Determiná la familia de complejidad temporal del siguiente método en el peor caso. Justificá analizando el comportamiento de los índices:

```
public static int buscarYValidar(int[] array) {  
    for(int i = 0; i < array.length; i++) { // N_1  
        if(array[i] == 5) { //C_1  
            return i; //C_2  
        } else {  
            for(int j = 0; j < i; j++) { // N_2  
                if(array[j] == 6) return i; //C_3  
            }  
        }  
    }  
    return -1; //C_4  
}
```

Respuesta:

$$(N_1 * C_1 * C_2) * (N_2 * C_3) * C_4 = O(n^2)$$

EJERCICIO 2:

Comparativa de "Fierros" (TDAs Estáticos)

Analizá la operación Acolar(x) en:

1. ColaPU (Estática): El nuevo elemento va siempre a la posición 0.
2. ColaPI (Estática): El nuevo elemento va al final del arreglo.
3. ColaLD (Dinámica): Usa nodos con punteros al primero y último. Indica el costo temporal para cada una y justifica por qué la implementación estática "PU" es distinta a las demás.

Respuesta:

En ColaPI y ColaLD los costos temporales son $O(1)$, ya que no se requiere ningún bucle para acolar o desacolar un elemento.

En ColaPU hay que desplazar todos los elementos a la derecha cada vez que se acola uno nuevo, para que este quede en la posición 0, esto requiere un bucle cuya duración depende del número de elementos, por lo tanto, $O(n)$.

EJERCICIO 3:

El algoritmo integrador filtrar: Un método recibe una Cola de n elementos, extrae todos para identificar los mayores a un "límite" (guardándolos en una cola nueva) y luego restaura la

original volcando los datos desde una estructura auxiliar tmp. Determina: (a) Complejidad temporal $O(n)$ y (b) Complejidad espacial $O(n)$.

Complejidad temporal:

1. Primer bucle: desacolar los n elementos $\rightarrow N \times O(1) = N$ operaciones.
2. Segundo bucle: restaurar desde tmp $\rightarrow N \times O(1) = N$ operaciones.
3. Los bucles son secuenciales, no anidados $\rightarrow$ se suman.
4. Total: $N + N = 2N \rightarrow O(n)$

Complejidad espacial:

- La cola auxiliar tmp guarda en algún momento los n elementos.
- La cola resultado, en el peor caso (todos son mayores al límite), también guarda hasta n elementos.
- Memoria adicional usada = proporcional a $n \rightarrow O(n)$

EJERCICIO 4:

Regla de la Suma (Código Secuencial)

Analiza el siguiente método y determina su familia de complejidad desglosando el costo de cada bloque independiente.

```
public static void procesoMisterioso(int[] arreglo) {  
  
    int n = arreglo.length;  
  
    System.out.println("Inicio");  
  
    for (int i = 0; i < n; i++) { arreglo[i] *= 2; }  
  
    for (int j = 0; j < n; j++) { System.out.println(arreglo[j]); }  
  
}
```

Regla de la Suma

Es analizar el código por bloques separados:

bloque a, bloque b, bloque c

Y el costo total es: $T(n) = T_A(n) + T_B(n) + T_C(n)$

Respuesta:

Bloque 1: `int n = arreglo.length;` $\rightarrow O(1)$

Bloque 2: `System.out.println("Inicio");` $\rightarrow O(1)$

Bloque 3: `for (int i = 0; i < n; i++) { -> O(n)`

```
        arreglo[i] *= 2;

    }
```

Bloque 4: `for (int j = 0; j < n; j++) { -> O(n)`

```
        System.out.println(arreglo[j]);

    }
```

Suma de Costos: $T(n) = O(1) + O(1) + O(n) + O(n)$

Se simplifican los costos, es decir se eliminan las constantes y nos queda **O(n)** es decir que la familia de complejidad es **O(n)** (Lineal)

EJERCICIO 5:

Regla del Producto (Matrices)

Analizá la complejidad de recorrer una matriz cuadrada $n \times n$ para buscar un número par. Identificá el peor caso.

Respuesta:

El peor caso es cuando no hay número par. La complejidad es $O(n^2)$, por $N_1 * N_2$.

EJERCICIO 6:

Jerarquía de Dominancia

Determiná la familia de complejidad total del siguiente bloque de código aplicando las reglas de simplificación.

```
public static void procesoMezclado(int[] datos) {
    int n = datos.length;
    // Bloque A: Bucle simple
    for (int i = 0; i < n; i++) { System.out.println(datos[i]); }
    // Bloque B: Bucles anidados
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) { datos[i] += datos[j]; }
    }
}
```

Respuesta:

- Bloque A (Bucle simple): Itera n veces ejecutando una instrucción de tiempo constante $O(1)$, lo que resulta en una complejidad parcial de $O(n)$.

- Bloque B (Bucles anidados): Consiste en un bucle dentro de otro, ambos iterando n veces con operaciones de tiempo constante. El número total de iteraciones es $n * n$, lo que da una complejidad parcial de $O(n^2)$.

Aplicación de las reglas de simplificación:

1. Regla de la suma: Dado que los bloques se ejecutan secuencialmente, sus tiempos se suman para obtener el tiempo total: $T(n) = O(n) + O(n^2)$
2. Jerarquía de dominancia: Al evaluar el límite del tiempo de ejecución cuando n crece, el término cuadrático (n^2) crece a un ritmo mucho mayor y domina al término lineal (n). Por lo tanto, el término de menor grado se descarta.

La familia de complejidad total del algoritmo se simplifica a $O(n^2)$.

EJERCICIO 7:

Divide y Vencerás

Explicá matemáticamente por qué la Búsqueda Binaria sobre un arreglo ordenado de n elementos pertenece a la familia $O(\log n)$. Utilizá el concepto de divisiones sucesivas de la tanda de datos.

Respuesta:

La búsqueda binaria pertenece a la familia $O(\log n)$ porque en cada ciclo, se descarta la mitad de los elementos del arreglo hasta llegar al elemento buscado. Es decir, el número de bucles necesarios es el $\log_2$ de la cantidad de elementos.

EJERCICIO 8:

El costo oculto del TDA Conjunto

Se desea calcular la intersección de dos Conjuntos (A y B) de tamaño n cada uno usando implementaciones estáticas. El algoritmo recorre A y por cada elemento llama a $B.Pertenece(x)$. Si es true, lo agrega a un conjunto resultado usando $Agregar(x)$. ¿Cuál es la complejidad temporal real considerando las validaciones internas del TDA?

Respuesta

La complejidad temporal real es $O(n^2)$. A simple vista parece un solo for de $O(n)$, pero los métodos del TDA esconden bucles adentro: $B.Pertenece(x)$ cuesta $O(n)$ porque recorre el conjunto, y $Agregar(x)$ también cuesta $O(n)$ porque valida que no haya duplicados. Por cada uno de los n elementos de A se hacen dos operaciones $O(n)$, dando $n \times O(n) = O(n^2)$. Si no se conoce la implementación interna del TDA, se pierde el verdadero costo del algoritmo.

EJERCICIO 9:

- Parámetros múltiples (Cola con Prioridad)
- Analizá la operación $Desacolar()$ en una Cola con Prioridad implementada con K subcolas.

- Justifica por qué su complejidad es $O(K)$ y por qué se dice que es independiente de la cantidad total de elementos n .

Respuesta:

Primero debemos recordar lo que es una cola con prioridad, que es una cola que organiza los elementos según niveles de prioridad tales como alta, media o baja. En la implementación común consiste en usar K subcolas. Le damos un valor a $K=3$ siendo 0 la mayor prioridad y para las demás $k-1$.

La operación **Desacolar()** debe buscar la primera subcola no vacía y sacar un elemento de esa subcola.

Ejemplo Conceptual:

```
for (int i = 0; i < K; i++) {  
    if (!subcolas[i].esVacia()) {  
        return subcolas[i].desacolar();  
    }  
}
```

Y ahora cuando se analizan los costos, se revisan las subcolas y no la cantidad total de elementos almacenados. En el peor caso debe revisar las K subcolas y por eso su complejidad es $O(K)$.

Su independencia de n viene porque el algoritmo no recorre todos los elementos almacenados y solo verifica si cada subcola está vacía o no. Es decir no importa si hay 10 elementos, 1000 elementos o 1 millón de elementos, **Desacolar()** sigue revisando únicamente las subcolas.

EJERCICIO 10:

Análisis de algoritmo con recursión doble

Dada la implementación recursiva de Fibonacci:

```
public static int fibonacci(int n) {  
    if (n <= 1) return n;  
    return fibonacci(n - 1) + fibonacci(n - 2);  
}
```

Sin realizar el cálculo matemático exacto, indicá a qué familia de costo (jerarquía) pertenece (¿Constante, Lineal o Exponencial?) y justificá basándote en el crecimiento de las llamadas.

Respuesta:

Es lineal, aunque cada vez que se llama a $\text{fibonacci}(n)$ se generan dos llamadas nuevas y las llamadas empiezan a multiplicarse muy rápido (como un árbol), la cantidad de "niveles" se mantiene proporcional a n , entonces el costo es $O(n)$. (2^n , es la cantidad de veces)